T6 Regresión logística binaria y multiclase

Índice

1. Introducción
2. Regresión logística binaria
3. Regresión logística (multiclase)

1 Introducción

Objetivo: introducir regresión logística binaria y multiclase

Clasificadores discriminativos: en oposición a los generativos, modelan **posteriors** directamente

$$p(y = c \mid \boldsymbol{x}, \boldsymbol{\theta}) = \cdots$$

Ajuste de discriminativos: **minimización de la neg-log-verosimilitud**, $\mathcal{L}(\boldsymbol{\theta}) = -\sum_n \log p(y_n \mid \boldsymbol{x}_n, \boldsymbol{\theta})$

- **Precisión y calibrado de posteriors:** suelen ser más precisos que los generativos, con posteriors mejor calibradas
- **Ingeniería de características:** facilitan el preproceso de características, típicamente mediante expansión polinómica
- **Redes neuronales:** las redes usuales pueden verse como extensiones de modelos discriminativos básicos

Regresión logística binaria: $C = 2$

Regresión logística (multiclase): $C \geq 2$

2 Regresión logística binaria

Regresión logística binaria: Bernoulli condicional para clasificación binaria, $y \in \{0, 1\}$,

$$p(y \mid \boldsymbol{x}, \boldsymbol{\theta}) = \mathrm{Ber}(y \mid \mu) \quad \text{con} \quad \mu = \sigma(a) \quad \text{y log-odds} \quad a = f(\boldsymbol{x}; \boldsymbol{\theta}) = \boldsymbol{w}^t \boldsymbol{x} + b$$

$$p(y = 1 \mid \boldsymbol{x}; \boldsymbol{\theta}) = \sigma(a) = \frac{1}{1 + e^{-a}}$$

$$p(y = 0 \mid \boldsymbol{x}; \boldsymbol{\theta}) = 1 - \sigma(a) = \sigma(-a) = \frac{1}{1 + e^{a}}$$

Con etiquetas $\tilde{y} \in \{-1, 1\}$: $\quad p(\tilde{y} \mid \boldsymbol{x}; \boldsymbol{\theta}) = \sigma(\tilde{y} a)$

Regla de decisión MAP: $\boldsymbol{x}$ se clasifica en la clase 1 si la log-odds es positiva

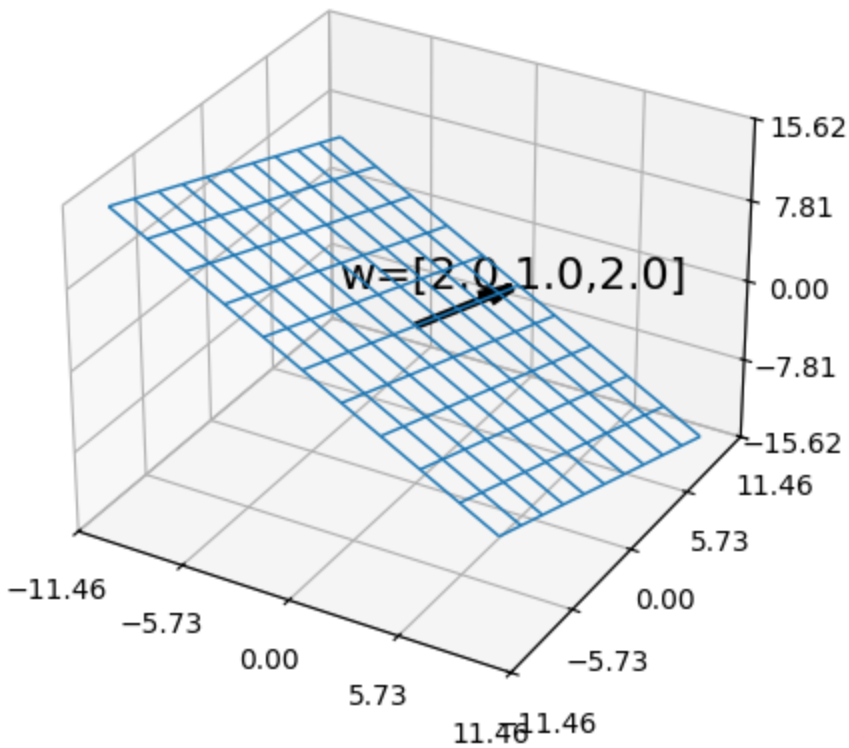
$$\mathbb{I}(p(y = 1 \mid \boldsymbol{x}) > p(y = 0 \mid \boldsymbol{x})) = \mathbb{I}\left(\log \frac{p(y = 1 \mid \boldsymbol{x})}{p(y = 0 \mid \boldsymbol{x})} > 0\right) = \mathbb{I}(a > 0)$$

Frontera MAP hiperplanar: $\quad \boldsymbol{w}^t \boldsymbol{x} + b = 0$

Notación homogénea: asume sesgo absorbido en $\boldsymbol{w}$ (con la ayuda de una dimensión de entrada extra e igual a 1)

Ejemplo de frontera: $\boldsymbol{w} = (2, 1, 2)^t$ y $b = 0$; frontera $2x_1 + x_2 + 2x_3 + 0 = 0$

In [ ]:
```
import numpy as np; import matplotlib.pyplot as plt
w1, w2, w3, b = 2.0, 1.0, 2.0, 0.0
x1, x2 = np.meshgrid(np.linspace(-10, 10, 11), np.linspace(-10, 10, 11))
x3 = lambda x1, x2: (-w1 * x1  - w2 * x2 - b) / w3
fig = plt.figure(figsize=(5, 5)); ax = fig.add_subplot(111, projection='3d')
ax.plot_wireframe(x1, x2, x3(x1, x2), rstride=1, cstride=1, linewidth=1)
scaw = 2.0; ax.quiver(0, 0, x3(0, 0), scaw * w1, scaw * w2, scaw * w3, linewidth=3, colors='black')
ax.text(scaw * w1, scaw * w2, scaw * w3, f"w=[{w1},{w2},{w3}]", fontsize=16, ha='center')
x_min, x_max = ax.get_xlim(); ax.set_xticks(np.linspace(x_min, x_max, 5))
y_min, y_max = ax.get_ylim(); ax.set_yticks(np.linspace(y_min, y_max, 5))
z_min, z_max = ax.get_zlim(); ax.set_zticks(np.linspace(z_min, z_max, 5));
```



Neg-log-verosimilitud (condicional): de $\boldsymbol{w}$ respecto a N datos $\mathcal{D} = \{(\boldsymbol{x}_n, y_n)\}$ (normalizada por N)

$$\mathrm{NLL}(\boldsymbol{w}) = -\frac{1}{N}\sum_{n=1}^{N} \log p(y_n \mid \boldsymbol{x}_n, \boldsymbol{w}) = -\frac{1}{N}\sum_{n=1}^{N} y_n \log \mu_n + (1-y_n)\log(1-\mu_n)$$

Gradiente (transpuesto): con $\boldsymbol{y} = (y_1, \ldots, y_N)^t$

$$\frac{\partial\, \mathrm{NLL}(\boldsymbol{w})}{\partial \boldsymbol{w}} = \frac{1}{N}\sum_{n}(\mu_n - y_n)\boldsymbol{x}_n^t = \frac{1}{N}(\boldsymbol{\sigma}(\mathbf{X}\boldsymbol{w}) - \boldsymbol{y})^t\mathbf{X}$$

Descenso por gradiente:

$$\begin{aligned}
\boldsymbol{w}_{i+1} &= \boldsymbol{w}_i - \eta_i \frac{1}{N}\mathbf{X}^t(\boldsymbol{\sigma}(\mathbf{X}\boldsymbol{w}_i) - \boldsymbol{y}) \\
\boldsymbol{w}_{i+1} &= \boldsymbol{w}_i - \eta_i \boldsymbol{x}_n(\sigma(\boldsymbol{x}_n^t\boldsymbol{w}_i) - y_n) \qquad \text{(para batch unitario)}
\end{aligned}$$

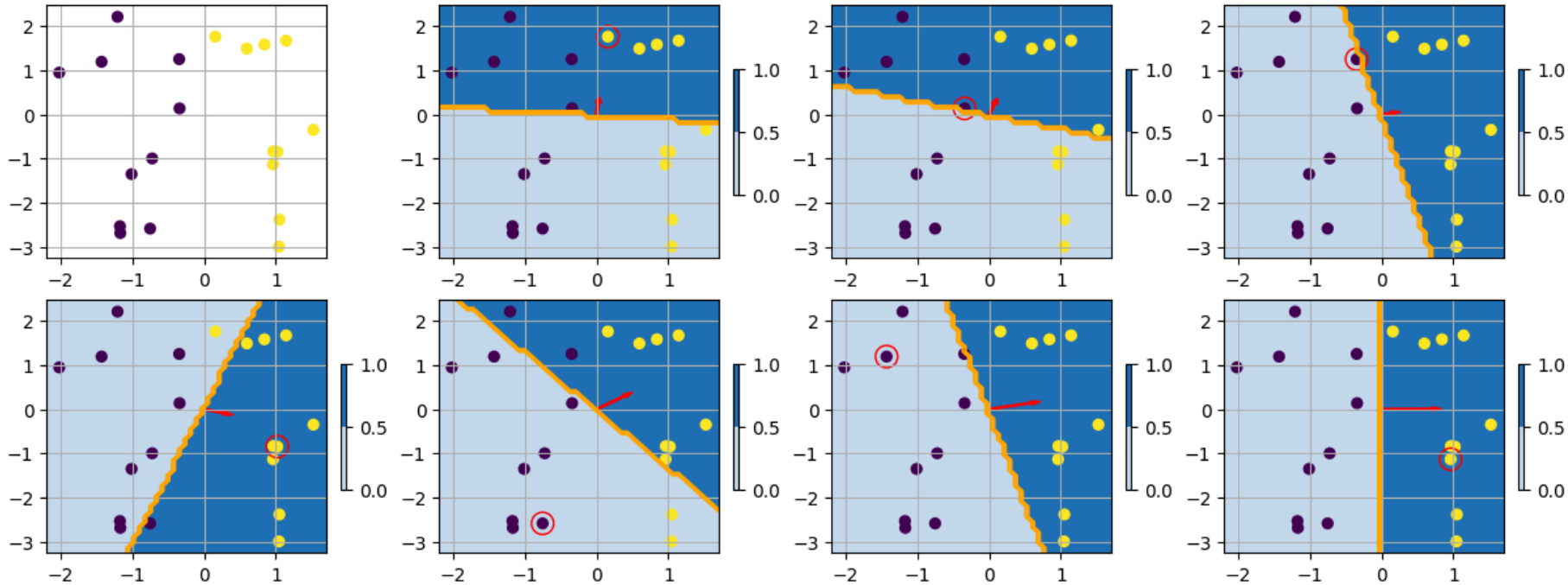
Ejemplo: $\mathbf{X} = [1,1,0;1,0,1;1,-1,0;1,0,-1]$ y $\boldsymbol{y} = (0,1,1,0)$ con sesgo absorbido en dimensión 1

In [ ]:
```
import numpy as np; np.set_printoptions(formatter={'float': lambda x: "{0:0.2f}".format(x)})
X = np.array([[1,1,0],[1,0,1],[1,-1,0],[1,0,-1]]); y = np.array([0,1,1,0]); eta = .01; w = np.zeros(3)
for i in range(100):
    w = w - eta / len(y) * X.T@(1/(1+np.exp(-X@w))-y)
print(w)
```

```
[0.00 -0.24 0.24]
```

Ejemplo: datos sintéticos 2d y modelo sin sesgo; $\boldsymbol{w}$ normal a la frontera, apunta a la clase 1

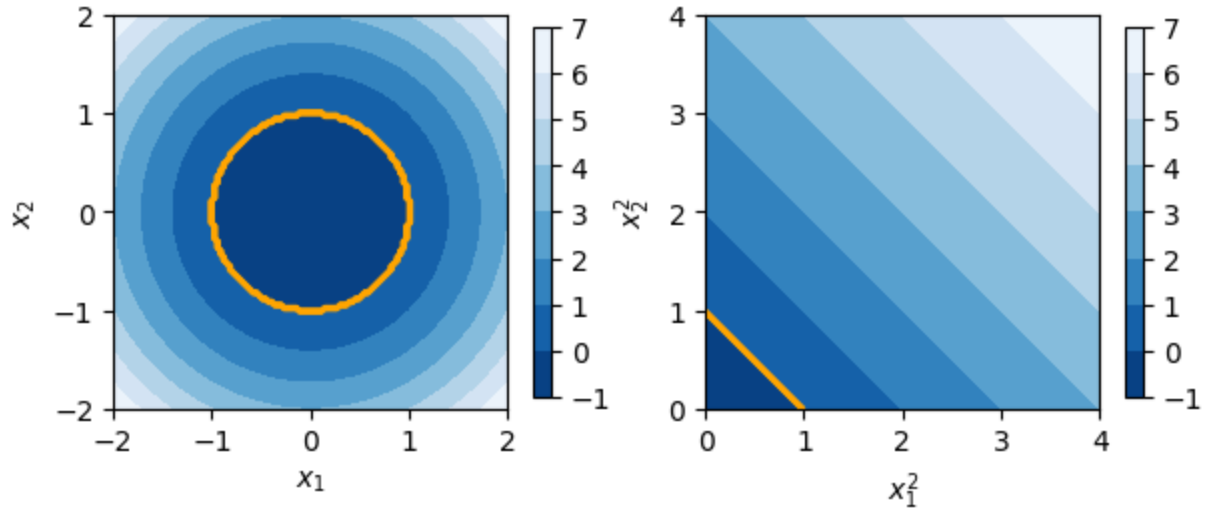
In [ ]:
```
import numpy as np; import matplotlib.pyplot as plt; from sklearn.datasets import make_classification
N, n_clusters_per_class, class_sep = 20, 2, 1.0
X, y = make_classification(n_samples=N, n_features=2, n_redundant=0, n_classes=2,
    n_clusters_per_class=n_clusters_per_class, class_sep=class_sep, random_state=1)
nrows = 2; ncols = 4; fig, axs = plt.subplots(nrows, ncols, figsize=(12, 4.5), constrained_layout=True)
ax = axs.flat[0]; ax.grid(); ax.scatter(*X.T, c=y, s=32)
x_min, x_max = ax.get_xlim(); y_min, y_max = ax.get_ylim()
xx, yy = np.meshgrid(np.linspace(x_min, x_max, 50), np.linspace(y_min, y_max, 50))
XX = np.c_[np.ravel(xx), np.ravel(yy)]; w = np.zeros(2); eta = 0.3
for n in np.arange(1, min(N, nrows * ncols)):
    mun = 1.0 / (1.0 + np.exp(- w @ X[n, :])); w = w - eta * X[n, :] * (mun - y[n])
    ax = axs.flat[n]; ax.grid()
    z = lambda x: w @ x; zz = np.heaviside(np.apply_along_axis(z, 1, XX), 0.0)
    ax.contour(xx, yy, zz.reshape(xx.shape), 1, colors='orange', linestyles='solid')
    cp = ax.contourf(xx, yy, zz.reshape(xx.shape), 1, cmap='Blues'); plt.colorbar(cp, ax=ax, shrink=0.5)
    ax.arrow(0, 0, *w, width=.03, facecolor='red', edgecolor='red')
    ax.scatter(*X.T, c=y, s=32); ax.scatter(X[n, 0], X[n, 1], facecolors='none', edgecolors='red', s=150)
```



Ingeniería de características: linearizar un problema con datos no linealmente separables mediante una función $\boldsymbol{\phi}(\boldsymbol{x})$

Ejemplo: $f(\boldsymbol{x}) = x_1^2 + x_2^2 - R^2 = \boldsymbol{w}^t\boldsymbol{\phi}(\boldsymbol{x}) + b$ con $\boldsymbol{\phi}(x_1, x_2) = (x_1^2, x_2^2)$, $\boldsymbol{w} = (1, 1)$ y $b = -R^2$

In [ ]:
```
import numpy as np; import matplotlib.pyplot as plt; R = 1
xr = np.linspace(-2, 2, 128); x1, x2 = np.meshgrid(xr, xr); X = np.c_[np.ravel(x1), np.ravel(x2)]
z = lambda x: x[0]**2 + x[1]**2 - R**2; Z = np.apply_along_axis(z, 1, X)
fig, axes = plt.subplots(1, 2, figsize=(7, 3)); axes[0].set(aspect='equal')
axes[0].set_xlabel('$x_1$', fontsize=10); axes[0].set_ylabel('$x_2$', fontsize=10)
axes[0].contour(x1, x2, (Z > 0).reshape(x1.shape), 8, colors='orange', linestyles='solid')
cp = axes[0].contourf(x1, x2, Z.reshape(x1.shape), 8, cmap='Blues_r')
plt.colorbar(cp, ax=axes[0], shrink=0.8); xxr = np.linspace(0, 4, 128)
xx1, xx2 = np.meshgrid(xxr, xxr); XX = np.c_[np.ravel(xx1), np.ravel(xx2)]
zz = lambda xx: xx[0] + xx[1] - R**2; ZZ = np.apply_along_axis(zz, 1, XX); axes[1].set(aspect='equal')
axes[1].set_xlabel('$x_1^2$', fontsize=10); axes[1].set_ylabel('$x_2^2$', fontsize=10)
axes[1].contour(xx1, xx2, (ZZ > 0).reshape(xx1.shape), 8, colors='orange', linestyles='solid')
cp = axes[1].contourf(xx1, xx2, ZZ.reshape(xx1.shape), 8, cmap='Blues_r')
plt.colorbar(cp, ax=axes[1], shrink=0.8);
```



3 Regresión logística

Regresión logística: categórica condicional para clasificación multiclase, $y \in \{1, \ldots, C\}$,

$$p(y \mid \boldsymbol{x}, \boldsymbol{\theta}) = \mathrm{Cat}(y \mid \boldsymbol{\mu}), \quad \boldsymbol{\mu} = \mathcal{S}(\boldsymbol{a}), \quad \text{logits } \boldsymbol{a} = f(\boldsymbol{x}; \boldsymbol{\theta}) = \mathbf{W}^t \boldsymbol{x} + \boldsymbol{b}, \quad \mathbf{W} \in \mathbb{R}^{D \times C},\ \boldsymbol{b} \in \mathbb{R}^C$$

Regla de decisión MAP: $\boldsymbol{x}$ se clasifica en la clase c si su logit es mayor que el resto

$$p(y = c \mid \boldsymbol{x}) > \max_{c' \neq c} p(y = c' \mid \boldsymbol{x}) \quad \Leftrightarrow \quad a_c > a_{c'} \quad \text{para todo} \quad c' \neq c$$

Región de decisión: intersección de $C - 1$ semi-espacios, esto es, un poliedro D-dimensional (acotado o no)

$$\mathcal{R}_c = \bigcup_{c' \neq c} \{\boldsymbol{x} \ : \ a_c > a_{c'}\} = \bigcup_{c' \neq c} \{\boldsymbol{x} \ : \ (\mathbf{W}_{:,c})^t \boldsymbol{x} > (\mathbf{W}_{:,c'})^t \boldsymbol{x}\}$$

Notación homogénea: asume sesgo absorbido en $\mathbf{W}$ (con la ayuda de una dimensión de entrada extra e igual a 1)

Neg-log-verosimilitud (condicional): de $\mathbf{W}$ respecto a N datos $\mathcal{D} = \{(\boldsymbol{x}_n, \boldsymbol{y}_n)\}$ con etiquetas de clase one-hot

$$\mathrm{NLL}(\mathbf{W}) = -\frac{1}{N}\sum_{n=1}^{N} \log p(\boldsymbol{y}_n \mid \boldsymbol{x}_n, \mathbf{W}) = -\frac{1}{N}\sum_{n=1}^{N}\sum_{c=1}^{C} y_{nc} \log \mu_{nc}$$

Gradiente (transpuesto): con $\boldsymbol{\mathcal{S}}$ aplicada por filas, $\mathbf{Y} = [\boldsymbol{y}_1, \ldots, \boldsymbol{y}_N]^t$

$$\frac{\partial \mathrm{NLL}(\mathbf{W})}{\partial \mathbf{W}} = \frac{1}{N}\sum_{n=1}^{N} (\boldsymbol{\mu}_n - \boldsymbol{y}_n)\boldsymbol{x}_n^t = \frac{1}{N}(\boldsymbol{\mathcal{S}}(\mathbf{X}\mathbf{W}) - \mathbf{Y})^t\mathbf{X}$$

Descenso por gradiente:

$$\begin{aligned}
\mathbf{W}_{i+1} &= \mathbf{W}_i - \eta_i \frac{1}{N}\mathbf{X}^t(\boldsymbol{\mathcal{S}}(\mathbf{X}\mathbf{W}_i) - \mathbf{Y}) \\
\mathbf{W}_{i+1} &= \mathbf{W}_i - \eta_i \boldsymbol{x}_n(\boldsymbol{\mathcal{S}}(\boldsymbol{x}_n^t\mathbf{W}_i) - \boldsymbol{y}_n^t) \qquad \text{(para batch unitario)}
\end{aligned}$$

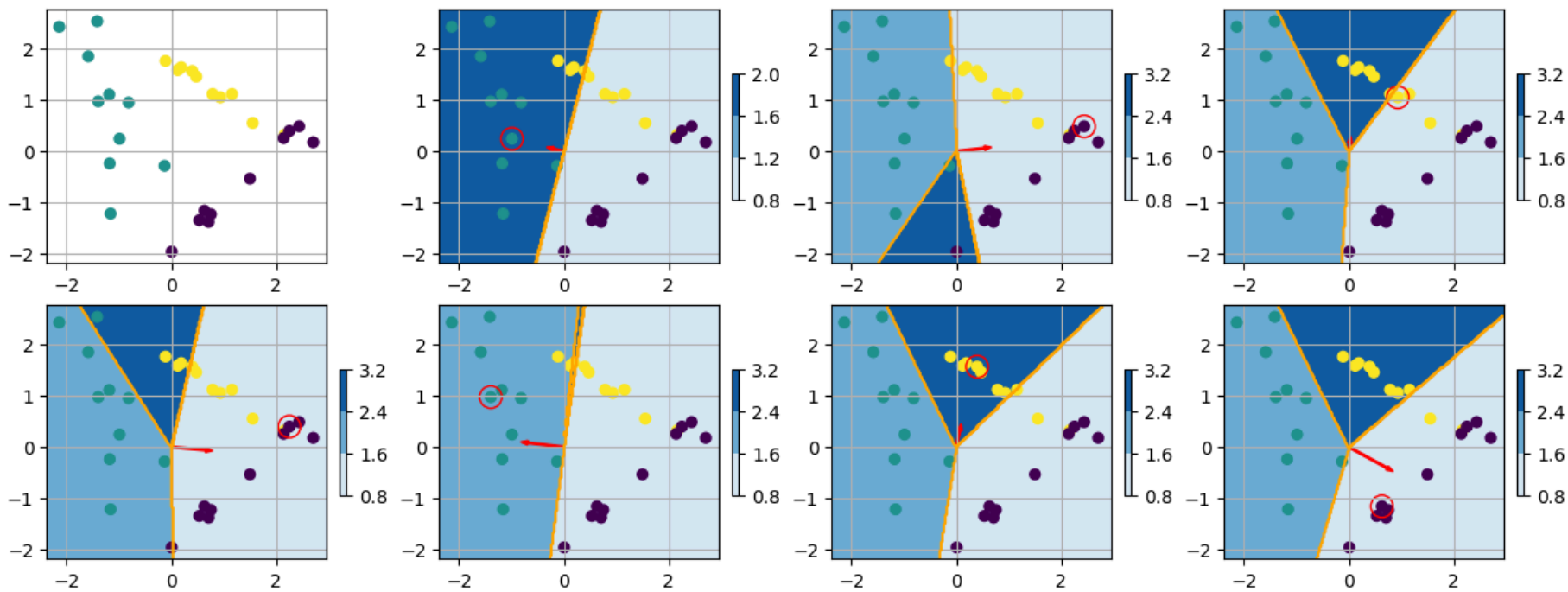
Ejemplo: $\mathbf{X} = [1,1,1;1,-1,1;1,-1,-1;1,1,-1]$ y $\boldsymbol{y} = (1,2,3,4)$; obtén $\hat{\mathbf{W}}$ con descenso por gradiente

In [ ]:
```
import numpy as np; np.set_printoptions(formatter={'float': lambda x: "{0:0.4f}".format(x)})
X = np.array([[1,1,1],[1,-1,1],[1,-1,-1],[1,1,-1]]); N = X.shape[0]
Y = np.array([[1,0,0,0],[0,1,0,0],[0,0,1,0],[0,0,0,1]]); eta = .01; W = np.zeros((3, 4))
for i in range(1000):
    Z = np.apply_along_axis(np.exp, 1, X @ W); Z = Z/Z.sum(axis=1, keepdims=True); Z -= Y
    W = W - eta / N * X.T@Z
print(W)
```

```
[[-0.0000 -0.0000 0.0000 -0.0000]
 [1.0891 -1.0891 -1.0891 1.0891]
 [1.0891 1.0891 -1.0891 -1.0891]]
```

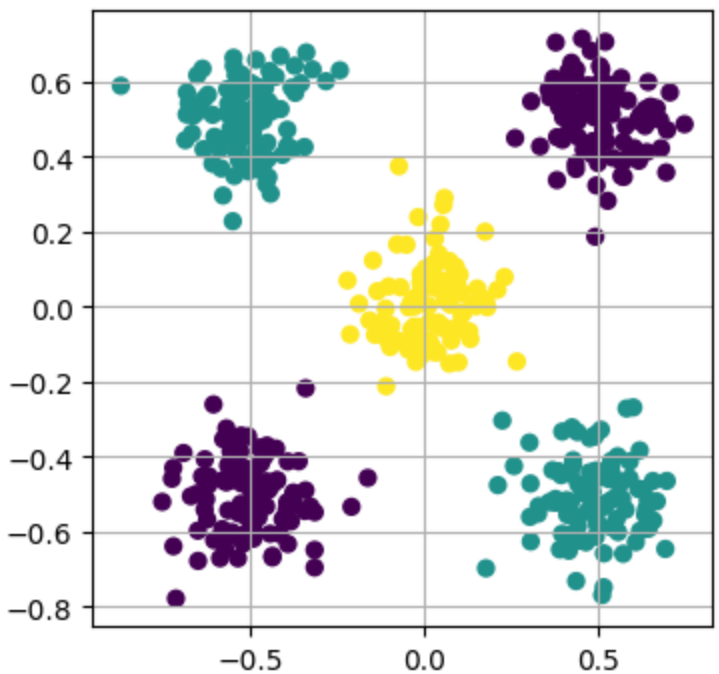
Ejemplo: datos sintéticos 2d y modelo sin sesgo; cada columna de $\mathbf{W}$ apunta a su clase

In [ ]:
```
import numpy as np; import matplotlib.pyplot as plt; from sklearn.datasets import make_classification
from scipy.special import logsumexp; N, n_clusters_per_class, class_sep = 30, 1, 1.0
X, y = make_classification(n_samples=N, n_features=2, n_redundant=0, n_classes=3,
    n_clusters_per_class=n_clusters_per_class, class_sep=class_sep, random_state=43)
nrows = 2; ncols = 4; fig, axs = plt.subplots(nrows, ncols, figsize=(12, 4.5), constrained_layout=True)
ax = axs.flat[0]; ax.grid(); ax.scatter(*X.T, c=y, s=32)
x_min, x_max = ax.get_xlim(); y_min, y_max = ax.get_ylim()
xx, yy = np.meshgrid(np.linspace(x_min, x_max, 200), np.linspace(y_min, y_max, 200))
XX = np.c_[np.ravel(xx), np.ravel(yy)]; W = np.zeros((2, 3)); eta = 0.3
for n in np.arange(1, min(N, nrows * ncols)):
    xn = X[n, :].reshape(-1, 1); an = (xn.T @ W).reshape(1, -1)
    mun = np.exp(an - logsumexp(an)); mun[0, y[n]] -= 1.0; W = W - eta * xn @ mun
    ax = axs.flat[n]; ax.grid()
    z = lambda x: np.argmax(x @ W) + 1; zz = np.apply_along_axis(z, 1, XX)
    ax.contour(xx, yy, zz.reshape(xx.shape), 1, colors='orange', linestyles='solid')
    cp = ax.contourf(xx, yy, zz.reshape(xx.shape), 2, cmap='Blues'); plt.colorbar(cp, ax=ax, shrink=0.5)
    ax.arrow(0, 0, *W[:,y[n]], width=.03, facecolor='red', edgecolor='red')
    ax.scatter(*X.T, c=y, s=32); ax.scatter(X[n, 0], X[n, 1], facecolors='none', edgecolors='red', s=150)
```

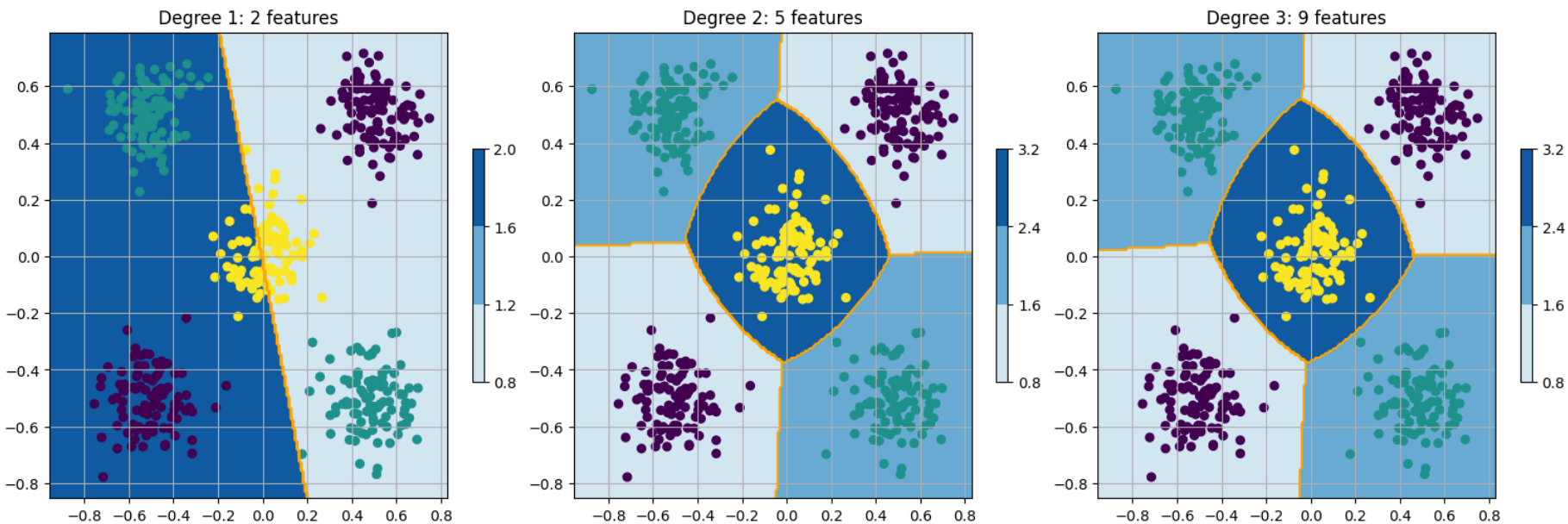


Ejemplo de ingeniería de características: $C = 3,\ \boldsymbol{x} = (x_1, x_2)^t$

In [ ]:
```
import numpy as np; import matplotlib.pyplot as plt; import matplotlib.colors as mcol
from scipy.stats import multivariate_normal as mvn
from sklearn.preprocessing import PolynomialFeatures; from sklearn.linear_model import LogisticRegression
np.random.seed(234); N, S = 100, 0.01 * np.eye(2)
Gs = [mvn([0.5, 0.5], S), mvn([-0.5, -0.5], S), mvn([0.5, -0.5], S), mvn([-0.5, 0.5], S), mvn([0, 0], S)]
X = np.concatenate([G.rvs(size=N) for G in Gs])
y = np.concatenate((1 * np.ones(N), 1 * np.ones(N), 2 * np.ones(N), 2 * np.ones(N), 3 * np.ones(N)))
fig, ax = plt.subplots(figsize=(4, 4)); ax.grid(); ax.scatter(*X.T, c=y, s=32)
x_min, x_max = ax.get_xlim(); y_min, y_max = ax.get_ylim()
xx, yy = np.meshgrid(np.linspace(x_min, x_max, 200), np.linspace(y_min, y_max, 200))
XX = np.c_[np.ravel(xx), np.ravel(yy)]
```



Características polinómicas de grados 1, 2 y 3; sin sesgo:

$$\phi(\boldsymbol{x}) = (x_1, x_2)^t \quad \phi(\boldsymbol{x}) = (x_1, x_2, x_1^2, x_1x_2, x_2^2)^t \quad \phi(\boldsymbol{x}) = (x_1, x_2, x_1^2, x_1x_2, x_2^2, x_1^3, x_1^2x_2, x_1x_2^2, x_2^3)^t$$

In [ ]:
```
degrees = [1, 2, 3]; nrows, ncols = 1, 3
C=1e4 # C = 1 / lambda: varianza del prior
fig, axes = plt.subplots(nrows, ncols, figsize=(15, 5), constrained_layout=True)
for i, degree in enumerate(degrees):
    ax = axes.flat[i]; ax.grid()
    transformer = PolynomialFeatures(degree, include_bias=False)
    X_poly = transformer.fit_transform(X)
    model = LogisticRegression(C=C, max_iter=1000).fit(X_poly, y)
    XX_poly = transformer.fit_transform(XX)
    zz = model.predict(XX_poly)
    ax.contour(xx, yy, zz.reshape(xx.shape), 1, colors='orange', linestyles='solid')
    cp = ax.contourf(xx, yy, zz.reshape(xx.shape), 2, cmap='Blues')
    plt.colorbar(cp, ax=ax, shrink=0.5)
    ax.scatter(*X.T, c=y, s=32)
    ax.set_title(f'Degree {degree}: {X_poly.shape[1]} features')
```